Research Square

Preprints are preliminary reports that have not undergone peer review.
They should not be considered conclusive, used to inform clinical practice,
or referenced by the media as validated information.

TCP BBR congestion control optimization algorithm for high fairness and low retransmission

Wansu Pan

wansupan@mail.ustc.edu.cn

Hefei Institutes of Physical Science, Chinese Academy of Sciences

Jinlin Xu

University of Science and Technology of China

Haibo Tan

Hefei Institutes of Physical Science, Chinese Academy of Sciences

Longle Cheng

University of Science and Technology of China

Xiru Li

Hefei Institutes of Physical Science, Chinese Academy of Sciences

Xiaofeng Li

Hefei Institutes of Physical Science, Chinese Academy of Sciences

Research Article

Keywords: congestion control, BBR, pacing gain, fairness, retransmission

Posted Date: September 26th, 2023

DOI: https://doi.org/10.21203/rs.3.rs-3365036/v1

License: This work is licensed under a Creative Commons Attribution 4.0 International License. Read Full License

Additional Declarations: No competing interests reported.

Abstract

Google proposed a new TCP congestion control algorithm based on Bottleneck Bandwidth and Round-trip propagation time (BBR). BBR achieves high transmission rate and low latency by estimating the available bottleneck capacity. Compared with the traditional loss-based congestion control algorithms such as CUBIC, BBR has obvious advantages in throughput. However, some studies have found that BBR has poor fairness among flows with different Round Trip Times (RTTs), inter-protocol unfairness and high packet loss rate in shallow buffer. BBR is already used in some cases, so its fairness and practicability improvements become very important. In this paper, the fairness of BBR is taken as the breakthrough point to deeply analyze the bandwidth probe mechanism. The pacing rate control model is derived and the effect of pacing gain on BBR performance is revealed. Furthermore, This paper proposes a BBR-PG algorithm, which adjusts pacing gain based on RTT to improve BBR. Simulation and real network experiments show that the proposed algorithm improves RTT fairness and inter-protocol fairness, reduces the retransmission rate,while retaining the throughput advantage of the original BBR.

1. Introduction

In 2016, Google[1] proposed a new congestion control algorithm (CCA) based on Bottleneck Bandwidth and Round-trip propagation time (BBR). BBR creates a network path model by measuring the available bottleneck bandwidth (BtlBw) and the Round-Trip propagation time (RTprop) to maximize delivery rate and minimize latency. The BBR implementation released by Google[2] shows that BBR can significantly improve throughput and bandwidth utilization compared to loss-based CCAs, such as CUBIC[3]. Due to its excellent performance, BBR has gained much attention ever since its release. Mathis and Mahdavi[4] claimed that BBR opens a new era in congestion control. However, some evaluations[5–7] found that the unfairness in sharing bottleneck between BBR and Reno/CUBIC. Literatures proposed that BBR has the problems of Round Trip Time (RTT) fairness[8–9] and high packet loss rate in shallow buffer[10–11]. Google is actively addressing these issues and has released the BBRv2 alpha[12]. Meanwhile, it is also hoped that more researchers will continue to optimize BBR.

BBR has been deployed in some application scenarios, but more research is needed to further improve the BBR to ensure that there are no more potential vulnerabilities. As a result of this motivation, this paper makes the following contributions:

1. The pacing rate control model is established and the origin of BBR fairness issues is revealed. Because the RTprop and queuing delay affect the original pacing gain, the RTT fairness and inter-protocol fairness of BBR are caused.
2. A optimization algorithm BBR-PG is proposed, which adaptively adjusts the original fixed pacing gain to balance the sending rates of different flows in the bottleneck queue.The phase time of the original BBR is optimized to accelerate the flows convergence.
3. BBR-PG is compared and analyzed by simulation and real network. The results show that BBR-PG maintains the throughput advantage of the original BBR, improves RTT fairness and inter-protocol fairness, and reduces the retransmission.

The rest of this article is arranged as follows. The Section 2 briefly introduces the principle of BBR and related optimization research. The theoretical model and derivation of the proposed algorithm are in Section 3. Section 4 is experimental results and analysis. Conclusions will be held in Section 5.

2. Related work

2.1 BBR congestion control

BBR controls congestion by limiting the pacing rate of packets and inflight to one BDP. BBR regards the maximum bandwidth (delivery rate) of the last 10 RTTs as the current BtlBw, and regards the minimum delay measured in the past 10 seconds as the current RTprop. The congestion window (CWND) and pacing rate were adjusted by *cwnd_gain* and *pacing_gain* as:

$$\text{pacing rate} = pacing_gain \times BtlBw \quad (1)$$

$$\text{CWND} = cwnd_gain \times \text{BDP} \quad (2)$$

BBR has four control phases: StartUP, Drain, ProbeBW and ProbeRTT. The Phases of BBR is shown in Fig. 1.

In the StartUP phase, the CWND and pacing rate will increase by setting *cwnd_gain* and *pacing_gain* as 2/ln2 (about 2.89). If the newly estimated bandwidth of three consecutive RTTs does not increase by at least 25%, BBR enters the Drain phase. BBR passes the *pacing_gain* is reduced to ln2/2 (about 0.35) to clear the remaining queue in the previous phase, *cwnd_gain* remains unchanged (2/ln2). In the ProbeBW phase, BBR has 8 cycles in the detection maximum bandwidth (*pacing_gain* [] = [1.25; 0.75; 1; 1; 1; 1; 1; 1; 1; 1]), and the duration of each *pacing_gain* was RTprop. If new RTprop is not sampled again within 10 seconds, BBR enters ProbeRTT. This phase is used to drain the buffer and obtain the minimum RTT.

2.2. Improvements of BBR

As BBR has gained wide attention, many researchers have evaluated the performance of BBR and conducted further optimization research to improve its defects. Some researchers have modified control parameters in BBR. With Tsunami[13] and BBRplus[14] receiving attention, but it is not clear whether either algorithm is applied to the actual program.

In terms of inter-protocol fairness, Hurtig et al.[15] evaluated BBR's inter-protocol fairness when the traffic consists of flows of different sizes. Zhang et al.[16] proposed modest BBR,which adjusted the rate according to the network situation, reduced the retransmission, and realized high throughput and inter-protocol fairness. Ware et al.[17] provided the first model capturing BBR's behavior in competition with loss-based CCAs. Song et al.[18] proposed BBR convergence window scaling (BBR-CWS), which adjusted the size of CWND through packet loss feedback. It reduced excessive retransmission and found a balance between algorithms. Google[19] proposed BBRv2 and released the preview version (called BBRv2 alpha), which can solve some fairness problems and limitations of the original BBR. Subsequently, Google[20] proposed a new BBR.Swift algorithm that utilizes the idea of delay as a congestion signal proposed by Kumar et al.[21] to achieve higher fairness and lower retransmission. Google continues to update the BBR, which still has RTT fairness and intra-protocol fairness problems to be further improved. In recent IETF-110[22], Google updated the BBR again. However, the BBR still has RTT fairness and intra-protocol fairness problems to be further improved.

In order to solve the RTT fairness problem, Ma et al. [8] proposed the BBQ algorithm. By setting the upper limit of the detection time, long RTT flows are prevented from preempting the bandwidth of short RTT flows. When no persistent queue is detected, BBQ reverts to the original BBR mechanism to maintain bandwidth detection speed. Yang et al. [23] proposed an Adaptive BBR algorithm, which alleviates the fairness between different RTT flows by changing the detection period and pacing gain of BBR. Kim et al. [24] proposed a method to suppress the establishment of bottleneck queues to enhance RTT fairness. Later, they proposed Delay-Aware BBR[25], which improved the fairness between different RTT flows. In our previous research, we also optimized RTT fairness by adjusting CWND [26] and sending rate[27] respectively to alleviate RTT unfairness.

BBR has been applied to different network scenarios to improve service quality[28–30]. When BBR is applied, some users may use these vulnerabilities to compete for bandwidth maliciously. Obviously, new methods are needed to further improve the fairness and practicability of BBR.

3. Proposed Work

In this section, the influences of pacing gain and convergence period on on BBR performance are studied by deriving the BBR pacing rate control model. Figure 2 shows the state machine and network path model in BBR. It can be observed that BBR mainly controls the transmission of data packets through two aspects: pacing rate and CWND. Therefore, in order to solve the complex problem of BBR, the pacing rate control model is established and derived to find the near-optimal solution.

3.1 BBR fluid model

Simplify the operation mechanism of BBR by establishing fluid model. Suppose that there are n flows with different flows passing through a bottleneck link with bandwidth C, where let $flow_i\ (i \in [1;\ n])$represents *flow*$_i$ and d_i(t) represents the delivery rate at time t. In the ideal state, $d_1 + d_2 + \ldots + d_n = C$. Let T_i(t) represent the RTT of *flow*$_i$ at time t, which is calculated by:

$$T_i\ (t) = T_{D_i}(t) + T_{p_i} = \frac{q_i\ (t)}{C} + T_{p_i}$$

3

Where T_D denotes queuing delay, q_i represents queue length, T_p represents RTprop.

Let I_i represent the inflight data of *flow*$_i$, namely, the upper bound of the bottleneck link, which is calculated by:

$$I_i(t) = d_i(t) \times T_i(t) = d_i(t) \times \left(\frac{q_i(t)}{C} + T_{p_i}\right)$$

4

The estimated BtlBw of the *flow*$_i$ at time t can be obtained:

$$BtlBw_i(\mathrm{t}) = \ max\ (d_i(\mathrm{t}))\ (T \in [t - 10RTT;\ t])$$

5

Combined with Eq. 4, *flow*$_i$ can obtain 1.25 times gain in the first probe-up cycle, so the maximum delivery rate at time t is shown as:

$$\max\left(d_i(t)\right) = \frac{\max\left(I_i(t)\right)}{T_i} = \frac{1.25 \times T_{p_i} \times BtlBw_i(t-\Delta t)}{T_i}$$

6

The detection cycle of BBR flow is 8RTprop, so the estimated BtlBw of *flow*$_i$ in the new round is updated as:

$$BtlBw_i(t) = \frac{1.25 \times T_{p_i} \times BtlBw_i(t-8T_{p_i})}{T_i}$$

7

Assuming that the total bandwidth is 1, the relationship between pacing gain and bandwidth is shown in Fig. 3, where A and B represent two different flows respectively. The total bandwidth, the initial bandwidths of the A flow and the B flow are assumed to be 1, *a* and 1-*a*, respectively. From Fig. 3, if A and B are respectively probe-up once, the bandwidth increment expressions of A and B can be obtained:

$$\Delta A = \frac{1-a}{0.25a+1} - \mathrm{a} = \frac{0.25a\left(1-a\right)}{0.25a+1}$$

8

$$\Delta B = \frac{1.25(1-a)}{1.25-0.25a} - (1-a) = \frac{0.25a(1-a)}{1.25-0.25a}$$

9

It can be seen from Eqs. 8 and 9 that the two flows intersect at *a* = 0.5 and are symmetric about *a* = 0.5. Obviously, the RTT of different flows on the same bottleneck link will not be exactly the same, and its initial bandwidth is difficult to achieve fair sharing.

The influence of RTT difference on bandwidth occupancy was further deduced. The RTT of *flow*$_1$ and *flow*$_2$ are set to T_1 and T_2 respectively, and $T_2 = \lambda T_1(\lambda \geqslant 1)$. The total inflight data in the time interval [*t*, *t* + *λ*] of the two flows can be calculated by Eqs. 10 and 11, where $\alpha = \lfloor (t-1) \times \lambda \rfloor$.

$$I_1\left(t\right) = \left[d_1\left(\alpha\right) \times \lambda t + d_1(\alpha-1) \times (1-\lambda t)\right] \times T_1$$

10

$$I_2\left(t\right) = d_2\left(t-1\right) \times T_2$$

11

The bandwidth occupation *oBw* of different flows can be determined by I_i:

$$oBw_i\left(t\right)=C\times\frac{I_i(\mathrm{t})}{\sum_{i=0}^{n}I_i(t)}$$

12

Substituting Eqs. 10 and 11 into Eq. 12, then:

$$oBw_2\left(t\right)=C\times\frac{I_2(t)}{I_2(\mathrm{t})+I_1(t)}=C\times\frac{\lambda\times d_2(t\text{-}1)}{\lambda\times d_2(t\text{-}1)+I_1(t)}$$

13

From Eq. 13, the bandwidth occupation of *flow*$_2$ is related to ratio λ. When d_i increases, oBw_2 also increases, which means that *flow*$_2$ preempts the bandwidth of *flow*$_1$. As a result, the BBR's throughput is affected by the RTT ratio.

From the above derivation, it can be seen that the long RTT flow has larger pacing gain in ProbeBW phase, resulting in a higher sending rate. If BBR misjudgment or other flows results in queuing delay T_D greater than $0.25T_p$ but less than T_p, the pacing gain will become negative and the sending rate will decrease instead of increasing. This is also the reason for bandwidth unfairness when BBR algorithm and other CCAs are deployed together. In addition, the stable convergence of BBR can be realized only when all flows stop increasing their sending rate. Therefore, the pacing gain is adaptively adjusted through RTT change to balance the sending rates of different flows.

3.2 The optimization algorithm: BBR-PG

In pacing-controlled phase, the actual pacing gain:

$$pg=\frac{1.25T_{p_i}}{T_i}=\frac{1.25T_{p_i}}{T_{D_i}+T_{p_i}}$$

14

It can be seen from Eq. 14 that the impact of the relationship between queuing delay and RTprop on the bandwidth. The BBR bandwidth detection process generally does not reach 1.25 times pacing gain unless the whole link is not queued at all. If the queuing time increases significantly, the *pg* will increase significantly with the increase of T_p. This is the root cause of RTT unfairness of BBR.

Let γ be the pacing gain regulatory factor, the bandwidths of the two flows with different RTT are the same in the ideal fair case, and the ratio of BDP is the ratio of RTT:

$$\frac{\gamma_1\times T_{p1}\times BtlBw_1(t)}{\gamma_2\times T_{p2}\times BtlBw_2(t)}=\frac{T_1}{T_2}=\frac{T_{p_1}+\mathrm{T}_D}{T_{p_2}+\mathrm{T}_D}$$

15

If *BtlBw*$_1$ = *BtlBw*$_2$, then:

$$\frac{\gamma_1 \times T_{p_1}}{\gamma_2 \times T_{p_2}} = \frac{T_{p_1} + \mathrm{T}_D}{T_{p_2} + \mathrm{T}_D}$$

16

The general term of γ is:

$$\gamma = g(T_p) = \frac{T_p + T_D}{T_p} = 1 + \frac{T_D}{T_p}$$

17

The regulatory factor *γ* is a subtractive function greater than 1 with T_p as the independent variable. The queuing delay T_D is equal to the difference between the actual RTT and RTprop, so γ can be obtained:

$$\gamma = \frac{T_p + sRTT\text{-}T_p}{T_p} = \frac{sRTT}{T_p}$$

18

Where *sRTT* represents the actual measured RTT, and its value is the average result of moving index of RTT.

After the adjusting of the γ, have the RTT fair bandwidth iterative equation :

$$BtlBw(t) = \frac{\gamma T_p}{T_p + T_D} \times BtlBw(t - 8T_p) = BtlBw(t - 8T_p)$$

19

It can be seen from Eq. 19 that the measurement bandwidth *BtlBw* does not increase and reaches equilibrium. In the γ expression g(T_p), the larger T_p, the smaller the pacing gain. Under this guarantee, different RTT flows can equal probe-up bandwidth.

In addition, the window of max-filtered bandwidth needs to be adjusted to match the fixed interval of steady cycle, so it is adjusted to:

$$WIN = \frac{T_{interval}}{T} + 4$$

20

Where *WIN* is the window of max-filtered bandwidth, $T_{interval}$ is set to 180ms in this paper, roughly equivalent to the 6 rounds for 30ms flows.

According to Eq. 20, it can be guaranteed that at least max-filtered bandwidth window can cover the interval between two probe-up cycles, and thus can be detected in max bandwidth window when probe-up. BBR-PG was implemented by modifying the *get_pacing_gains* () and *bbr_update_bw* () functions in the BBR source code. The pseudocode of ProbeBW phase in BBR-PG is shown in Algorithm 1.

Algorithm 1: ProbeBW phase in BBR-PG

Input: *rtt_us* /**RTT**/, *min_rtt_us* /**Minimum RTT**/, *pacing_gain*

Output: *pacing_gain*

Initialization: $T_{interval=}$180ms, *rtt_cnt* = 0

1: bbr_update_min_rtt() /*Track min RTT seen in the min_rtt filter window*/

2: if bbr->mode = = ProbeBW then

3: *get_pacing_gains* ()

4: if bbr->cycle_idx = = 0 then

5: $\gamma = rtt_us * rtt_us$

6: *pacing_gain = pacing_gain* × γ /*Update pacing_gain*/

7: return *pacing_gain*

8: *bbr_update_bw* () /*Update BtlBw */

9: rtt_cnt++

10: if BtlBw > = bbr_max_bw then

11: $WIN = T_{interval} * 4$

12: *minmax_running_max* (*WIN*, *rtt_cnt*) /*Incorporate new sample into BtlBw filter.*/

13: end if

14: end if

Next, the inter-protocol fairness optimization when BBR and CUBIC coexist is discussed. In addition to the above *pg* interference, the cycle of ProbeRTT phase also affects inter-protocol fairness. BBR enters the ProbeRTT phase at a fixed interval of 10s to remeasure RTprop, due to the existence of CUBIC, the measurement of RTprop will be inaccurate. When the BBR flows enters the ProbeRTT phase, the active detection of bandwidth by the CUBIC flows will fill the queue. Since the queue is not empty at this time, this causes the measured RTprop of the BBR flows is larger than the actual RTprop. Larger RTprop will result in larger CWND, resulting in more inflight, which will benefit BBR flows. ws are measured with lower RTprop than before. This process is repeated throughout the data transmission time, and continuous oscillations can lead to unfairness between protocols, regardless of whether the two flows achieve a fair average throughput.

This problem is solved by randomizing the time to enter the ProbeRTT phase. The fixed interval of 10 seconds in the original BBR mechanism is changed to a random interval with a mean of 10 seconds as the next time to enter ProbeRTT phase. The time interval adopted in this paper is shown:

$$T_{rand}=rand()\backslash\% \ (h_2 - h_1 + 1) + h_1$$

21

Where T_{rand} is the random interval time, h_1 and h_2 are set to 5 and 15 respectively, which means the value range is set from 5s to 15s.

By random intervals, different BBR flows no longer enter the ProbeRTT phase at the same time. Some BBR flows enter the ProbeRTT phase and measure RTprop through the empty queue. Other BBR/CUBIC flows that are not in ProbeRTT phase can actively compete for the free bandwidth in 200ms. In this way, BBR-PG sacrifices a small amount of bandwidth to alleviate aggression and retransmission, thus achieving better fairness.

4. Experiments evaluation

4.1 Testbed setup

This section uses NS3, LAN and WAN networks to evaluate the performance of the proposed algorithm BBR-PG. the experimental topology is shown in Fig. 4. NS3 is a network emulator that creates a virtual network running multiple hosts, links, and switches on a single machine. Based on the BBR implementation framework[31], a large number of simulation experiments have been carried out on NS3, where the condition variables include bottleneck bandwidth, RTT and buffer size, etc.

In order to eliminate the uncontrollable interference in the network (e.g., background traffic, special token bucket policy program, AQM deployment, etc.), a LAN network is formed by four Linux hosts, and set flows sharing bottleneck scenario. NetEm was used to simulate the channel part, and different network environments were configured.

The WAN experiment was constructed by using 4 Linux physical hosts and 2 cloud servers in the laboratory (node1 in London, UK and Node2 in Silicon Valley, USA). The bandwidth of each server is 5Mbps, and the latency is about 300ms and 230ms, respectively. The Linux5.4 kernel module is deployed in the whole cluster, and sch_fq is loaded to adjust the traffic according to BBR requirements.

Moreover, in order to quantify the fairness under different buffer sizes, Jain fairness index is introduced[32]. Jain fairness Index can be used to measure fairness in resource competition. The calculation method using Eq. 22, where n represents n sender and x_i is throughput of the i link.

$$J = \frac{(\sum_{i=1}^{n} x_i)^2}{n \sum_{i=1}^{n} {x_i}^2}$$

22

Jain fairness index (here in after referred to as fairness index) can well reflect the difference in throughput, with values ranging from [0,1]. The closer the fairness index is to 1, the better the fairness of bandwidth allocation is.

4.2 Transmission speed

The Web service is set up on the server via Nginx, and 100Mbps files are placed for download. The BBR, BBQ, BBR-PG and CUBIC were tested in the same time period in each experiment. The experiment is divided into single-thread and multi-thread tests, and the statistical results are given in Fig. 5. The download speed of BBR、BBQ and BBR-PG are basically the same, and the three algorithms have a significant effect on improving the transmission speed. The 5.4 kernel default CUBIC is the slowest. In the experimental process, we found that the speed of BBQ would increase rapidly after the beginning, and then the speed would decrease significantly, which could be seen

in many experiments. On the other hand, BBR and BBR-PG download speeds are relatively stable. At the beginning, the download speed rose rapidly, then fell back slightly, and finally remained stable. In Fig. 5(b), the speed of several algorithms has been improved, compared with Fig. 5(a). However, the lifting effect of CUBIC is not obvious in long delay link (Node2). Overall, BBR□BBQ and BBR-PG have little difference in speed improvement, which improves at least 40% compared with CUBIC.

4.3 RTT fairness

The experiment was set up under 0.5BDP and 5BDP buffers, and 10ms RTT flow competed with 50ms RTT flow for bottleneck bandwidth. Figure 6 shows the throughput comparison results. For BBR and BBQ, when the buffer increases, the throughput difference between flows increases obviously. Although the throughput of BBR-PG fluctuates greatly, the throughput difference is reduced. Among the four algorithms, the fairness of BBR-PG is the best.

Next, the fairness of the four algorithms under different buffer sizes is compared. In 0.1BDP ~ 100BDP buffer, average throughput and fairness index of 10ms RTT flows and 50ms RTT flows are provided in Fig. 7. The throughput difference becomes larger with the increase of buffer size. In general, BBR-PG has better RTT fairness than the other three algorithms, especially in the deep buffer. Compared with BBR, the fairness index increased by about 53%.

In addition to buffer size has a significant impact on RTT fairness, RTT differences also significantly affect RTT fairness. As depicted in Figs. 8(a)(b)(c), with the increase of RTT difference, the throughput of long RTT flow gradually dominates, and the fairness index gradually decreases. The fairness index of BBR is only 0.59. but that of BBR-PG can be maintained at about 0.95. In Fig. 8(d), the RTT fairness of CUBIC deteriorates with the increase of RTT ratio, and the final fairness index is about 0.89. On the whole, BBR-PG can maintain high fairness among different RTT ratios. Especially when the RTT ratio is greater than 4, the fairness index of BBR-PG is the highest, which improves by 61% compared with BBR.

Then experiments were carried out on LAN. For each test, the number of flows varies from 2 to 4. Figure 9 shows the statistical results of the throughput ratio of 10ms and 50ms RTT flows.

As observed from the results provided in Fig. 9, 50ms RTT flows have obvious advantages in different buffer sizes. Although the disadvantage of 10ms RTT flows is alleviated in shallow buffers, their bandwidth share is still far below its reasonable share. Different from the other three algorithms, 10ms RTT flow of CUBIC is superior in shallow buffer. Compared with BBR, the fairness of BBR-PG improved by 11%(0.8BDP) and 23%(8BDP) respectively. This further proves the effectiveness of the simulation results and the proposed algorithm.

Meanwhile, RTT fairness test was carried out in WAN to verify the performance of each algorithm. As we can see from Fig. 10, the experiment results of WAN were similar to those of NS3 and LAN. The share of throughput between 50ms RTT flow and 10ms RTT flow in WAN also has a certain deviation. Different from the NS3 and LAN experiment results, the results in Fig. 10(b) show a different trend from Fig. 9(b). In the deep buffer, the throughput share of 50ms RTT flow does not increase significantly. The bandwidth sharing bias of two flows in WAN is less than that of NS3 and LAN experiments.

4.4 Inter-protocol fairness

Firstly, the inter-protocol fairness experiments are carried out on NS3. The experiment was divided into two groups, which carried out bandwidth competition for 2 flows and 100 flows respectively. The bottleneck bandwidth is 100Mbps and the RTT is 10ms. The throughput of several algorithms is shown in Fig. 11.

As shown in Figs. 11 (a)(c)(e), with the buffer increases, the throughput advantage of the algorithm will be reversed, and the CUBIC will gradually dominate. The results of 100 flows competition plotted in Figs. 11(b)(d)(f), CUBIC is always at a disadvantage. Regardless of single flow competition or multi-flow competition, the fairness index of BBR-PG is much better than that of BBR. In the shallow buffer, and the fairness index increases by 46% and 54% respectively, indicating that BBR-PG can improve the inter-protocol fairness.

Next, several experiments were conducted in LAN environment to compare the throughput ratio of algorithm co-existing with CUBIC. Figure 12 shows the comparison results of bandwidth occupation.

BBR, BBQ and BBR-PG algorithms all show bandwidth preemption advantages under different bottleneck bandwidth. Sometimes BBR gets an almost fair share of bottleneck bandwidth (Low bandwidth; Low latency), while at other times (High bandwidth; High latency), most of the bottleneck bandwidth is occupied by BBR flow. This is because CUBIC, as a loss-based CCA, tends to fill the bottleneck buffer until it is exhausted no matter how large the buffer is. Compared with Fig. 12(a) and 12(b), the preemption of BBR-PG becomes smaller, and the inter-protocol fairness under different bandwidth conditions is better than CUBIC.

4.5 Retransmission

The experiments are conducted to verify the influence of different buffer sizes and the number of contended flows on the retransmission rate. The sender uses different algorithms to transmit single or multiple flows to the receiver, and the buffer size is set to 0.1BDP or 1BDP. The retransmission rate is shown in Fig. 13. The starting point is the retransmission rate of a single 10ms RTT flow.

As shown in Fig. 13, the retransmission rate of BBR is significantly higher than that of the other three algorithms. With the increase of the number of flows, a large amount of retransmission occurs in all four algorithms. The retransmission rate of BBR-PG is much lower than BBR and BBQ, and slightly 0.5% higher than that of CUBIC. The reason why the retransmission rate of BBR-PG is higher than CUBIC may be that the aggressive detection bandwidth brought by pacing gain in BBR-PG results in a certain amount of data packet retransmission, which needs to be further optimized in future work.

The retransmission rates of the four algorithms were then tested on LAN and WAN networks. The retransmission rates of the four algorithms are shown in Fig. 14.Similar to Fig. 13, the retransmission rate of BBR is significantly higher than that of the other three algorithms. When there were 50 flows, the retransmission rate of BBR was about 12.7%, while the retransmission rate of BBQ and BBR-PG remained around 4.7% and 4.6%. This indicates that BBR is increasing throughput at the expense of high packet retransmission in shallow buffers. BBR may not be a good choice if the content passed is sensitive to packet loss. In this case, users need to make a trade-off between throughput and quality of experience. Overall, the retransmission of BBR-PG was much lower than that of BBR and 0.4% higher than that of CUBIC. It is proved that BBR-PG can reduce retransmission and improve performance.

5. Conclusions

In this paper, we analyzed and exposed the critical reasons for RTT fairness and inter-protocol fairness problems in BBR. Then, we established the pacing rate control model to analyze the change of actual pacing gain, and found that RTT measurement led to the inaccurate evaluation of BtlBw. According to the pacing gain mechanism, an improved BBR-PG is proposed. BBR-PG adaptively adjusts the pacing rate by increasing the regulator factor γ instead of the fixed 1.25 or 0.75. Meanwhile, the phase time of ProbeRTT phase is randomized.

Different network test results show that BBR-PG can effectively alleviate the fairness problem in BBR and retain the advantage of BBR in throughput. In terms of RTT fairness, BBR-PG had the best fairness index. In NS3 simulation network, its fairness index is 50% higher than BBR, and RTT fairness of LAN and WAN networks is also improved. In the inter-protocol fairness tests, the fairness index of BBR-PG in NS3 simulation network improves by more than 46% compared with BBR. The LAN network test results also verify the fairness of BBR-PG. The retransmission test results show that BBR-PG can effectively reduce the retransmission of BBR in shallow buffer. Overall, BBR-PG not only alleviates the fairness problem of BBR, but also reduces the retransmission.

In the future work, we will continue to optimize the parameters of the BBR-PG algorithm, such as $T_{interval}$ and T_{rand}. We plan to use reinforcement learning to predict network load so that $T_{interval}$ or T_{rand} can be adjusted adaptively. In addition, with the development of BBRv2 version, the improved method can be applied to the latest BBR to solve its problem.

Declarations

Funding This work was partly supported by National key research and development program, “blockchain” key special project (2021YFB2700700).

Data Availability statement No data were used to support this article.

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Cardwell N, Cheng Y, Gunn CS, Yeganeh SH, Jacobson V (2016) Bbr: Congestion-based congestion control: Measuring bottleneck bandwidth and round-trip propagation time. Queue 14(5):20–53. https://doi.org/10.1145/3012426.3022184
2. Cardwell N, Cheng Y, Gunn CS, Yeganeh SH, Jacobson V (2017) BBR: congestion-based congestion control. Commun ACM 60(2):58–66. https://dl.acm.org/doi/fullHtml/10.1145/3009824
3. Ha S, Rhee I, Xu L (2008) CUBIC: a new TCP-friendly high-speed TCP variant. ACM SIGOPS operating systems review. 42(5):64–74. https://doi.org/10.1145/1400097.1400105
4. Mathis M, Mahdavi J (2019) Deprecating the TCP macroscopic model. ACM SIGCOMM Computer Communication Review 49(5):63–68. https://doi.org/10.1145/3371934.3371956
5. Hock M, Bless R, Zitterbart M (2017) Experimental evaluation of BBR congestion control. 2017 IEEE 25th international conference on network protocols (ICNP), 1–10. Toronto, Canada. https://doi.org/10.1109/ICNP.2017.8117540
6. Miyazawa K, Sasaki K, Oda N, Yamaguchi S (2018) Cyclic performance fluctuation of TCP BBR. 2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC), 811–812. Tokyo, Japan.

https://doi.org/10.1109/COMPSAC.2018.00132

7. Cao Y, Jain A, Sharma K, Balasubramanian A, Gandhi A (2019) When to use and when not to use BBR: An empirical analysis and evaluation study. Proceedings of the Internet Measurement Conference. 130–136. Amsterdam, Netherlands. https://doi.org/10.1145/3355369.3355579
8. Ma S, Jiang J, Wang W, Li B (2017) Fairness of congestion-based congestion control: Experimental evaluation and analysis. arXiv preprint. https://doi.org/10.48550/arXiv.1706.09115. arXiv:1706.09115
9. Scholz D, Jaeger B, Schwaighofer L, Raumer D, Geyer F, Carle G (2018) Towards a deeper understanding of TCP BBR congestion control. 2018 IFIP networking conference (IFIP networking) and workshops. 1–9. Zurich, Switzerland. https://doi.org/10.23919/IFIPNetworking.2018.8696830
10. Sasaki K, Hanai M, Miyazawa K, Kobayashi A, Oda N, Yamaguchi S (2018) TCP fairness among modern TCP congestion control algorithms including TCP BBR. 2018 IEEE 7th international conference on cloud networking (CloudNet). 1–4. Tokyo, Japan. https://doi.org/10.1109/CloudNet.2018.8549505
11. Zhang H, Zhu H, Xia Y, Zhang ZL, Deng Y (2019) Y. Performance analysis of BBR congestion control protocol based on NS3. 2019 Seventh International Conference on Advanced Cloud and Big Data (CBD). 363–368. Suzhou, China. https://doi.org/10.1109/CBD.2019.00071
12. Google (2018) TCP BBRv2 Alpha/Preview. Retrieved from https://github.com/google/bbr/tree/v2alpha
13. Tsunami (2017) Retrieved from https://github.com/dlxg/Linux-NetSpeed &gt
14. BBRplus (2017) Retrieved from https://blog.csdn.net/dog250/article/details/80629551 &gt
15. Hurtig P, Haile H, Grinnemo KJ, Brunstrom A, Atxutegi E, Liberal F, Arvidsson A (2018) Impact of TCP BBR on CUBIC traffic: A mixed workload evaluation. In 2018 30th International Teletraffic Congress (ITC 30), vol 1. Vienna, Austria, pp 218–226. https://doi.org/10.1109/ITC30.2018.00040
16. Zhang Y, Cui L, Tso FP (2018) Modest BBR: Enabling better fairness for BBR congestion control. 2018 IEEE symposium on computers and communications (ISCC). 00646–00651. Natal, Brazil. https://doi.org/10.1109/ISCC.2018.8538521
17. Ware R, Mukerjee MK, Seshan S, Sherry J (2019) Modeling bbr's interactions with loss-based congestion control. Proceedings of the internet measurement conference. 137–143. Amsterdam, Netherlands. https://doi.org/10.1145/3355369.3355604
18. Song YJ, Kim GH, Cho YZ (2020) BBR-CWS: Improving the inter-protocol fairness of BBR. Electronics 9(5):862. https://doi.org/10.3390/electronics9050862
19. Cardwell N, Cheng Y, Yeganeh SH, Swett I, Vasiliev V, Jha P, Seung Y, Mathis M, Jacobson V (2019) BBRv2: A model-based congestion control. Proceedings of the IETF 105th Meeting, 1–21. Montreal, Canada. Retrieved from https://datatracker.ietf.org/meeting/105/materials/slides-105-iccrg-bbr-v2-a-model-based-congestion-control-00 &gt
20. Cardwell N, Cheng Y, Yang K, Yeganeh SH, Jha P, Seung Y, Hsiao L, Mathis M, Jacobson V BBR update: 1: BBR., Swift (2020) ; 2: Scalable loss handling. *Proceedings of the IETF 109th Meeting*, 1–21. Online. Retrieved from https://datatracker.ietf.org/meeting/109/materials/slides-109-iccrg-update-on-bbrv2-00 &gt
21. Kumar G, Dukkipati N, Jang K (2020) Swift: Delay is simple and effective for congestion control in the datacenter. Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication. 514–528. Virtual Event, USA. https://doi.org/10.1145/3387514.3406591

22. Cardwell N, Cheng Y, Yeganeh SH et al (2021) BBR Updates:Internal Deployment, Code, Draft Plans, Proceedings of the IETF 110th Meeting, 1–8
23. Yang M, Yang P, Wen C, Liu Q, Luo J, Yu L (2019) Adaptive-BBR: Fine-grained congestion control with improved fairness and low latency. IEEE Wireless Communications and Networking Conference (WCNC). 1–6. Marrakesh, Morocco. https://doi.org/10.1109/WCNC.2019.8885527
24. Kim GH, Mahmud I, Cho YZ (2019) Fairness improvement of BBR congestion control algorithm for different RTT flows. 2019 International Conference on Electronics, Information, and Communication (ICEIC), 1–2. Auckland, New Zealand. https://doi.org/10.23919/ELINFOCOM.2019.8706412
25. Kim GH, Cho YZ (2019) Delay-aware BBR congestion control algorithm for RTT fairness improvement. IEEE Access. 8https://doi.org/4099-4109
26. Pan WS, Tan HB, Li XF, Li XR (2021a) Improved RTT Fairness of BBR Congestion Control Algorithm based on Adaptive Congestion Window. Electronics 10(5):615. https://doi.org/10.3390/electronics10050615
27. Pan WS, Li XF, Tan HB, Xu JL, Li X (2021b) Improvement of RTT Fairness Problem in BBR Congestion Control Algorithm by Gamma Correction. Sensors 21(12):4128. https://doi.org/10.3390/s21124128
28. Parichehreh A, Alfredsson S, Brunstrom A (2018) Measurement analysis of TCP congestion control algorithms in LTE uplink. 2018 Network Traffic Measurement and Analysis Conference (TMA), 1–8. Vienna, Austria.https://doi.org/10.23919/TMA.2018.8506522
29. Atxutegi E, Liberal F, Haile HK, Grinnemo KJ, Brunstrom A, Arvidsson A (2018) On the use of TCP BBR in cellular networks. IEEE Commun Mag 56(3):172–179. https://doi.org/10.1109/MCOM.2018.1700725
30. Najmuddin S, Asim M, Munir K, Baker T, Guo Z, Ranjan R (2020) A BBR-based congestion control for delay-sensitive real-time applications. Computing 102(12):2541–2563. https://doi.org/10.1007/s00607-020-00829-2
31. Claypool M, Chung JW, Li F (2018) BBR' an implementation of bottleneck bandwidth and round-trip time congestion control for ns-3. Proceedings of the 10th workshop on ns-3, 1–8. New York, United States https://doi.org/10.1145/3199902.3199903
32. Jain RK, Chiu DMW, Hawe WR (1984) A quantitative measure of fairness and discrimination. Eastern Research Laboratory, Digital Equipment Corporation. Hudson, MA, 21. Retrieved from https://ocw.cs.pub.ro/courses/_media/isrm/laboratoare/new/a_quantitative_measure_of_fairness_and_d.pdf

Figures

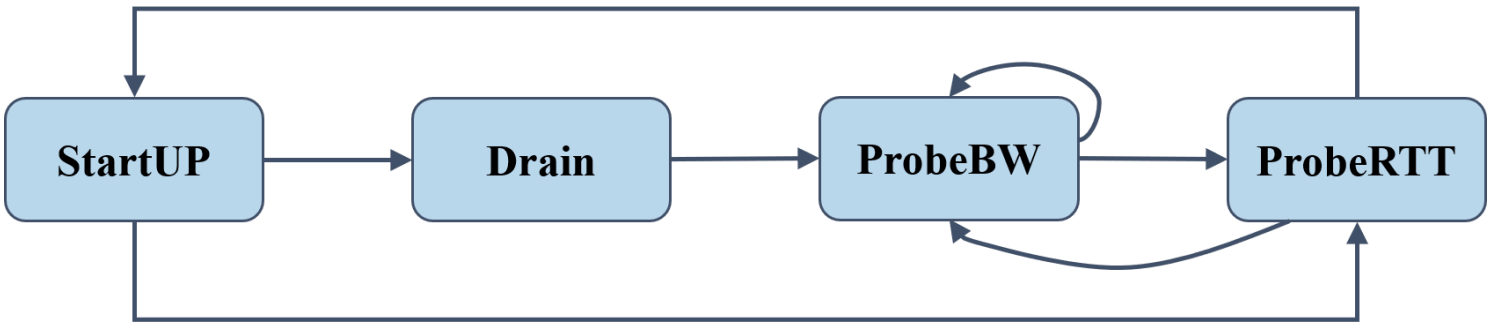


Figure 1

The phasesof BBR

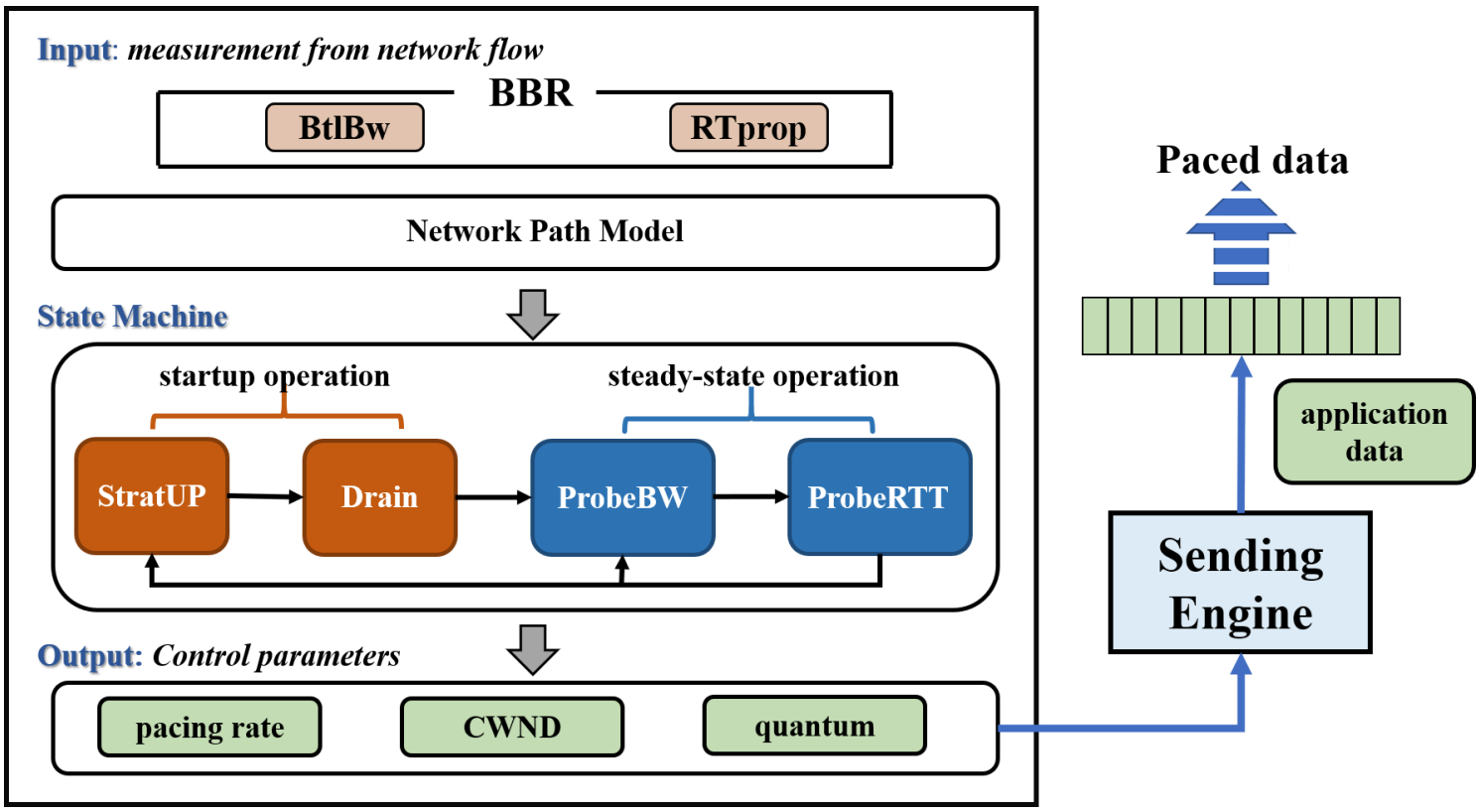


Figure 2

The state machine and network path model

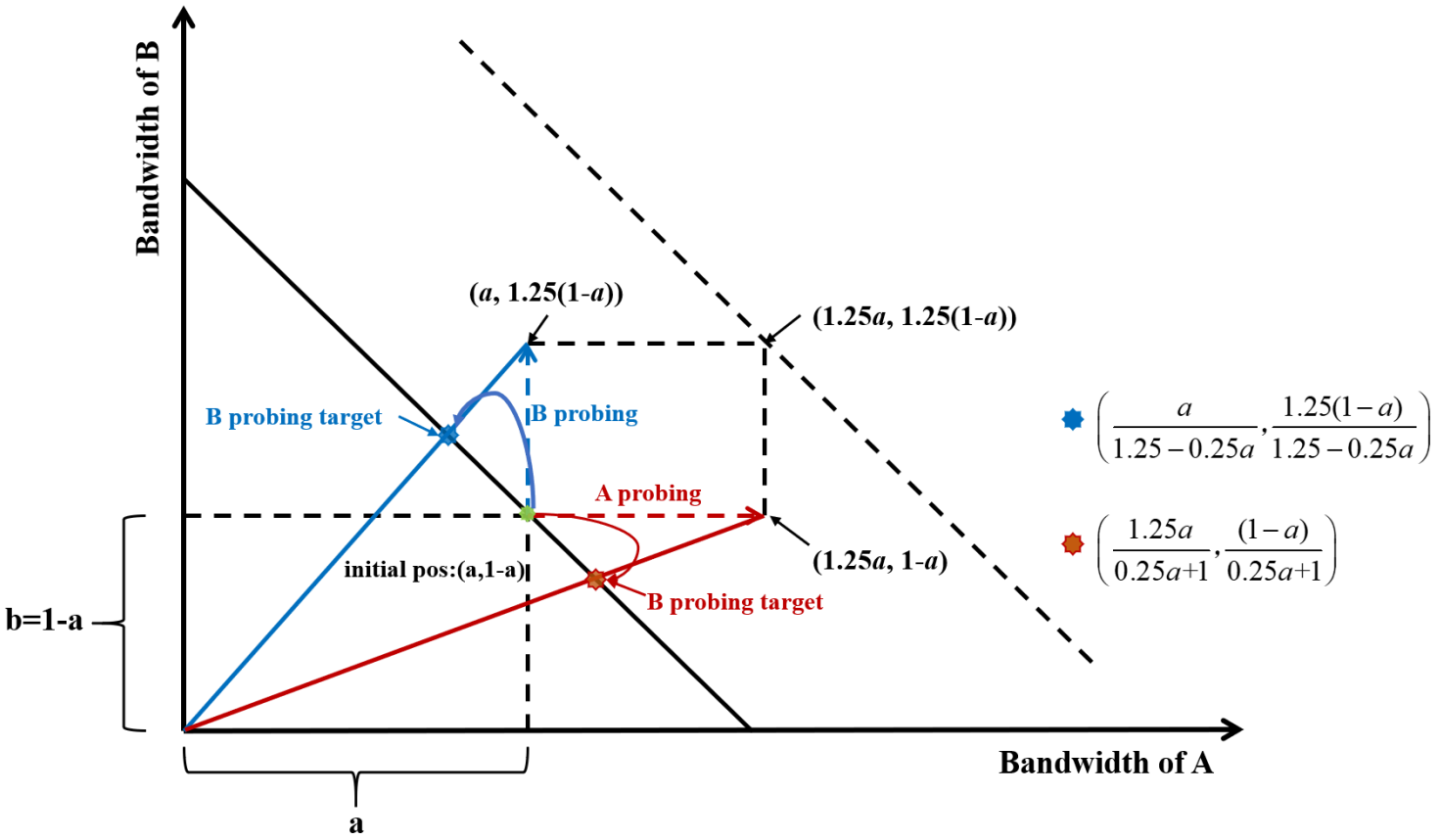
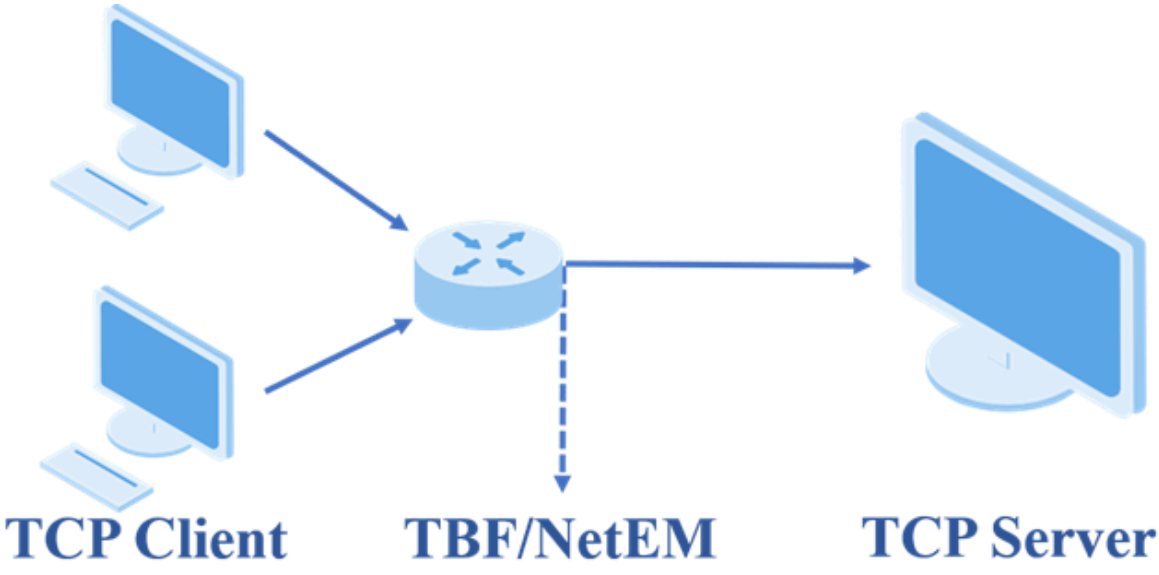
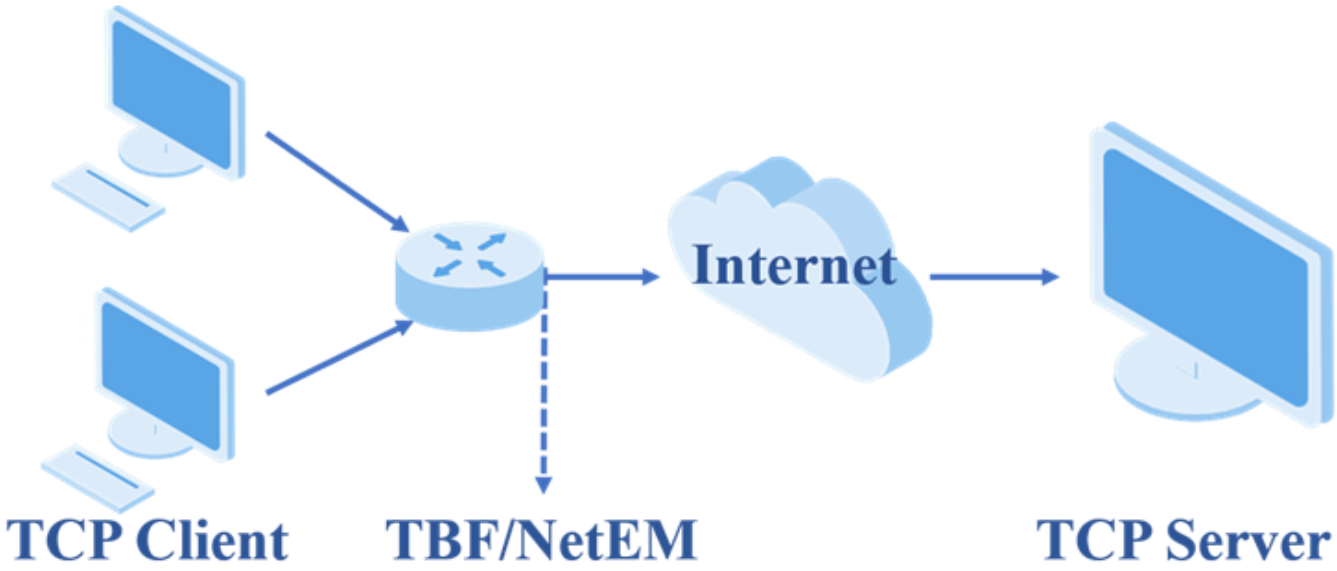


Figure 3

The relationship between pacing gain and bandwidth.



(a) NS3/LAN topology



(b) WAN topology

Figure 4

Experimental topology of the network

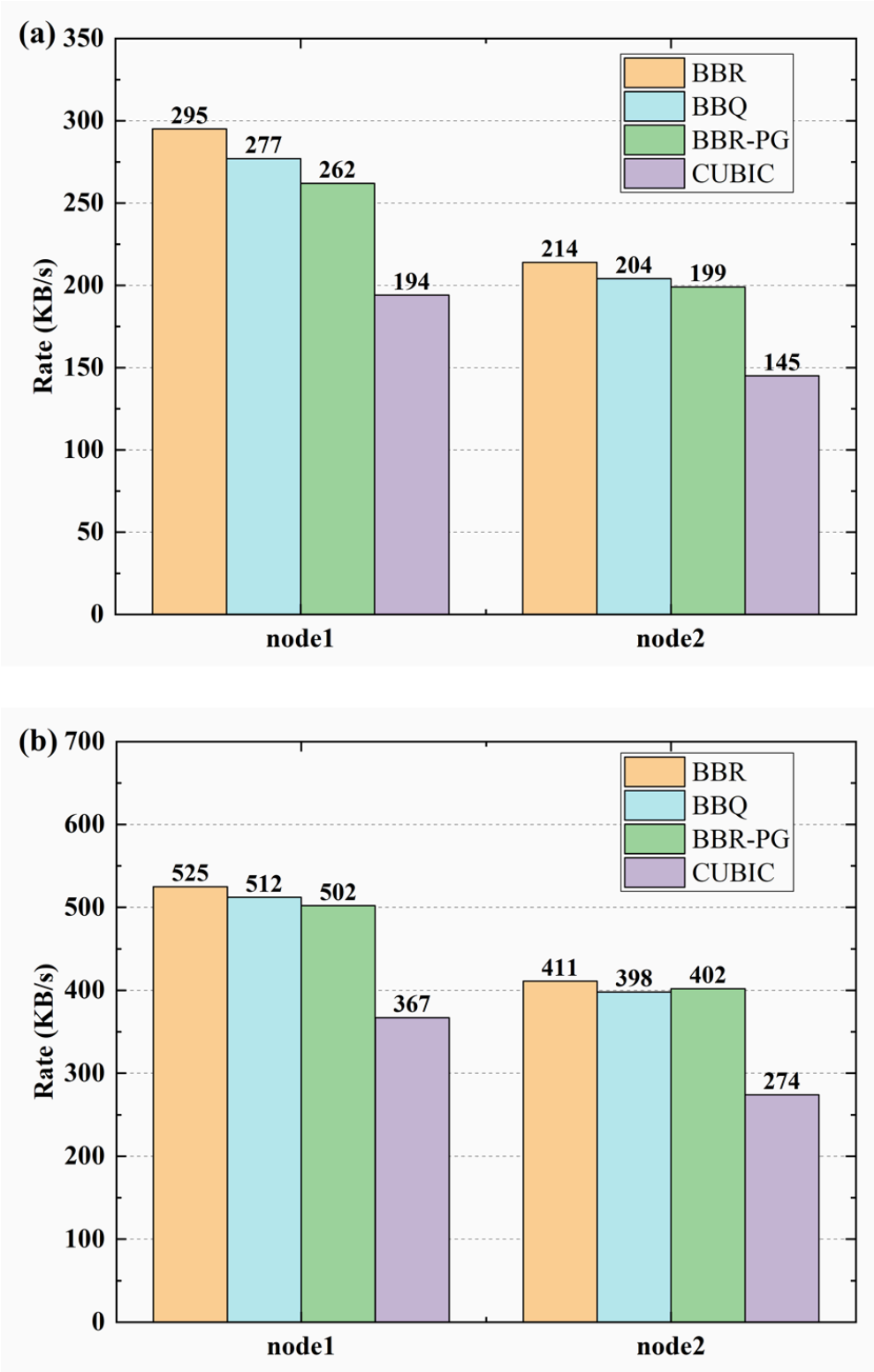
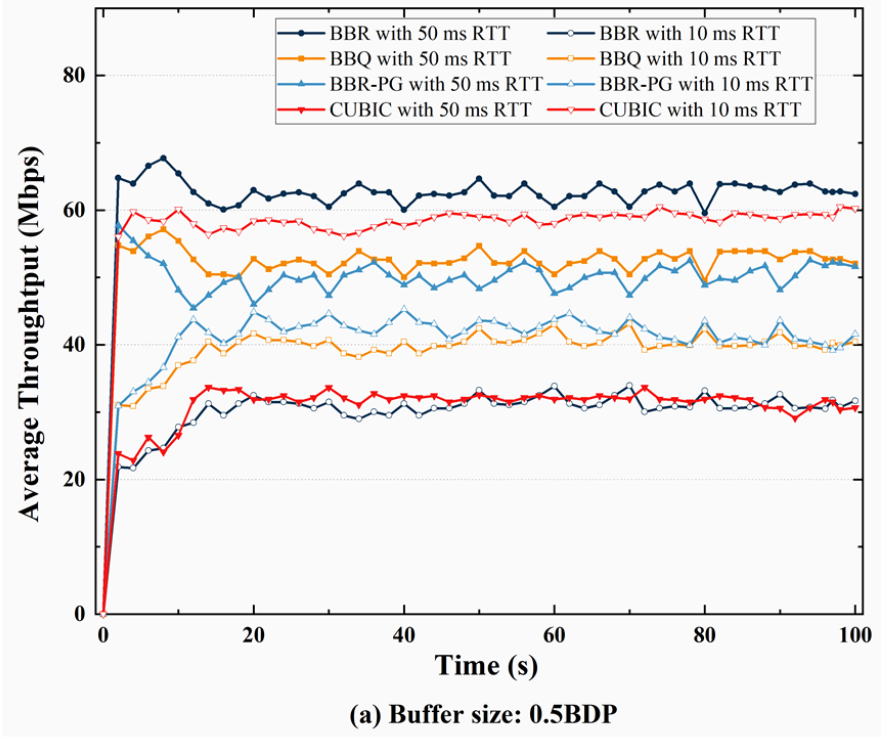
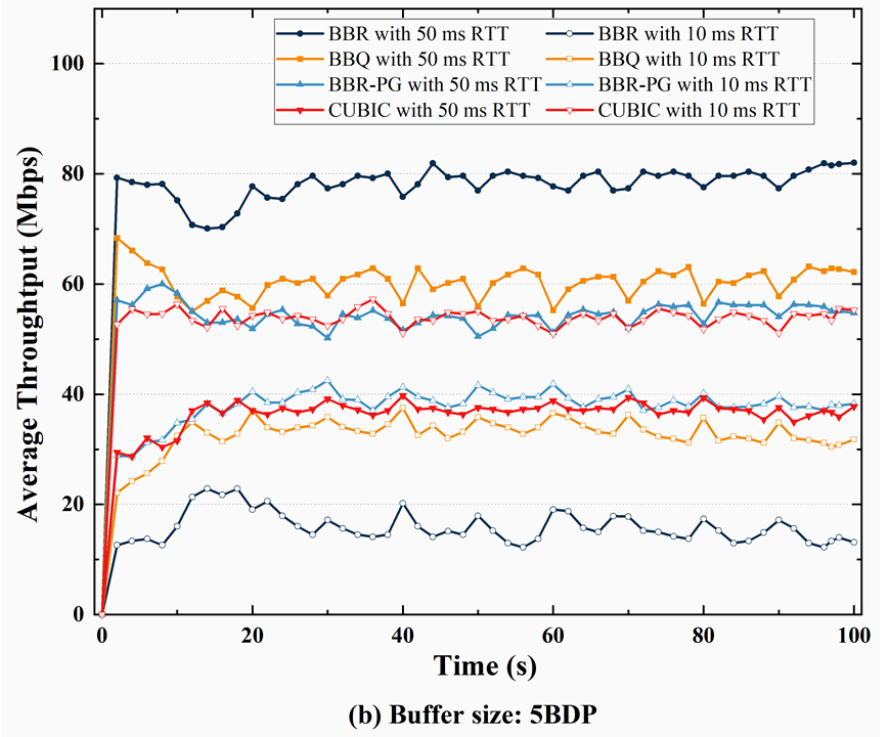


Figure 5

Transmission speed of each algorithm



(a) Buffer size: 0.5BDP



(b) Buffer size: 5BDP

Figure 6

Average throughput comparison of 10ms RTT flows and 50ms RTT flows

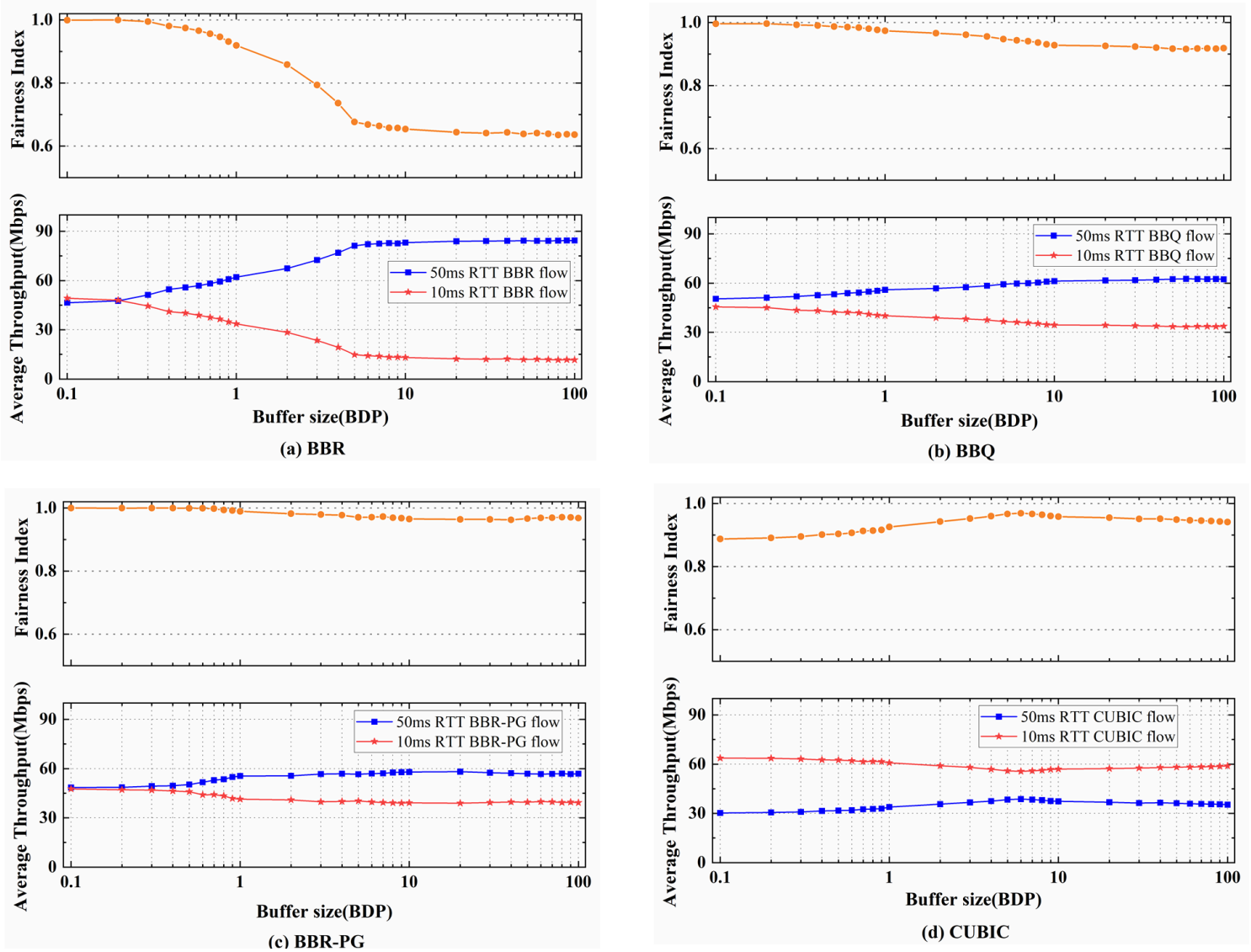


Figure 7

Average throughput and fairness index of 10ms RTT flows competing with 50ms RTT flows in different buffer sizes

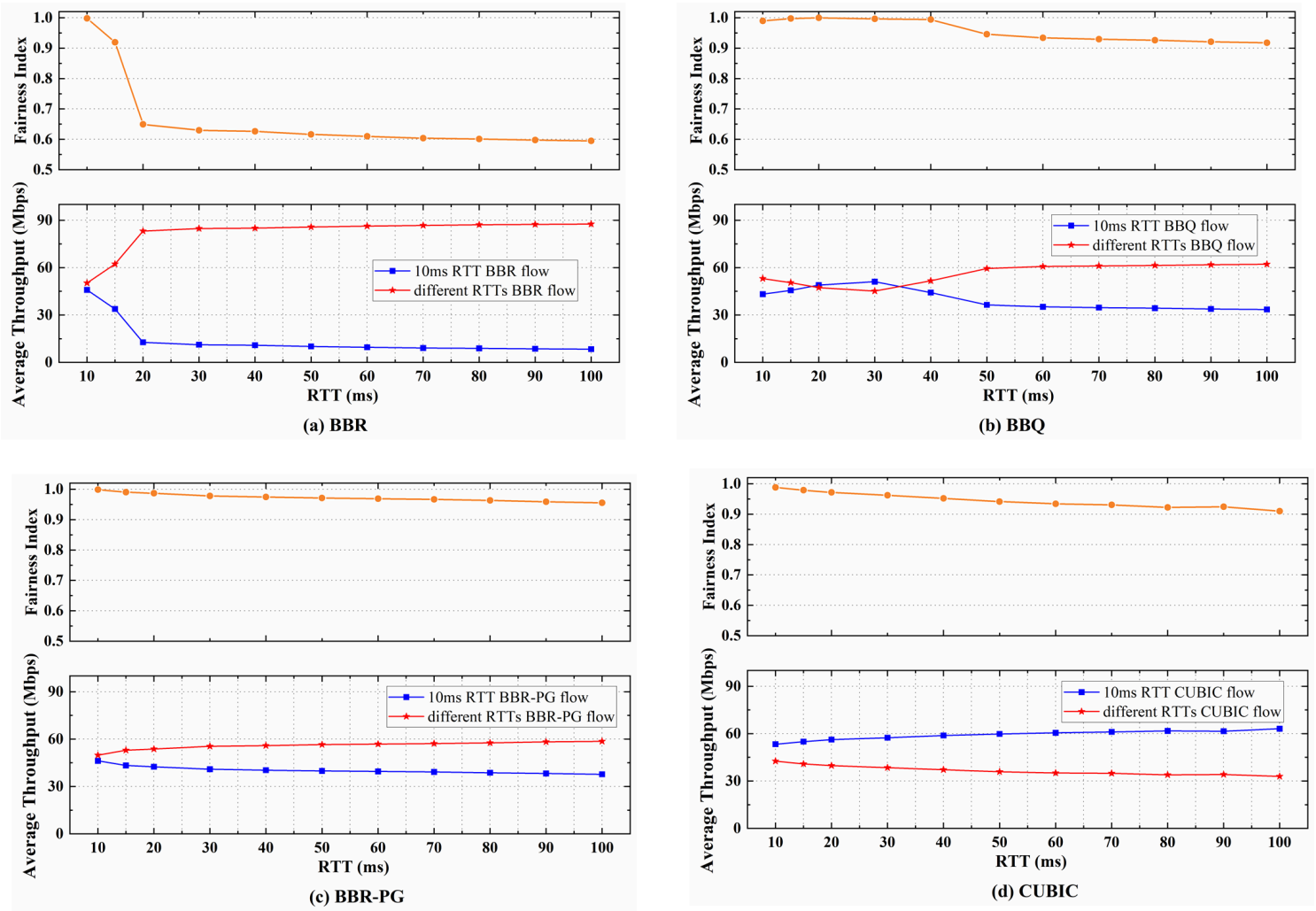


Figure 8

Average throughput and fairness index of 10ms RTT flows coexisting with different RTT flows in 5BDP buffer

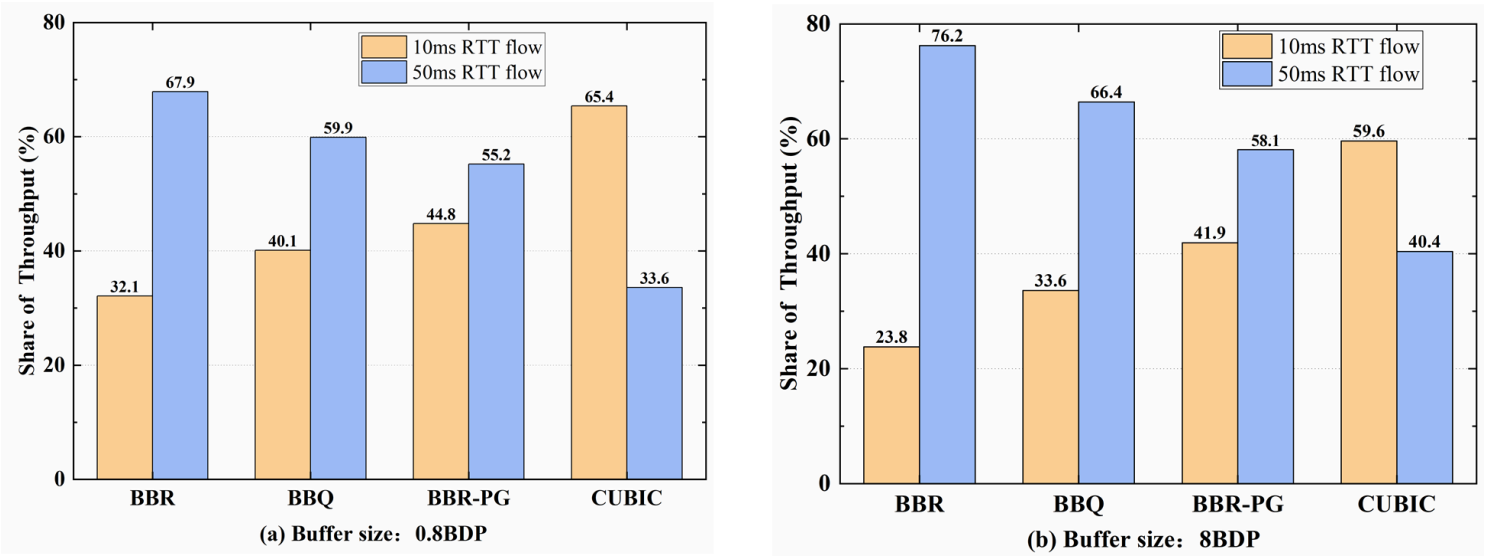


Figure 9

Throughput shares in LAN network

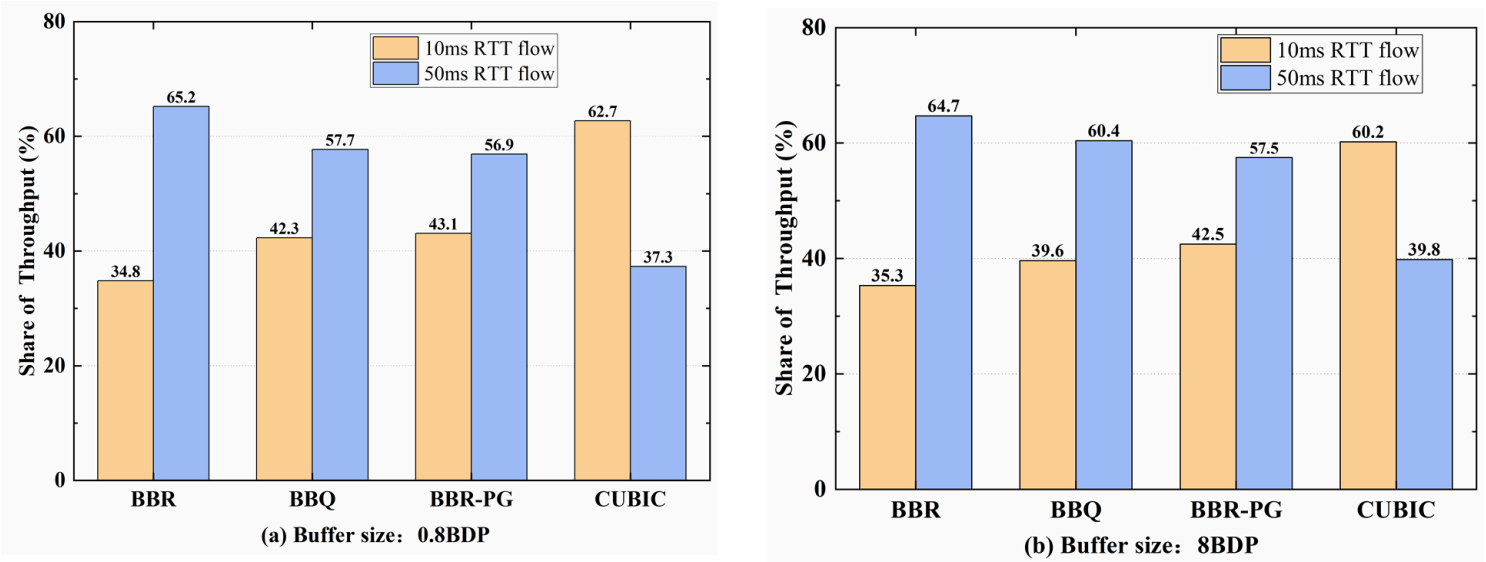


Figure 10

Throughput shares in WAN network

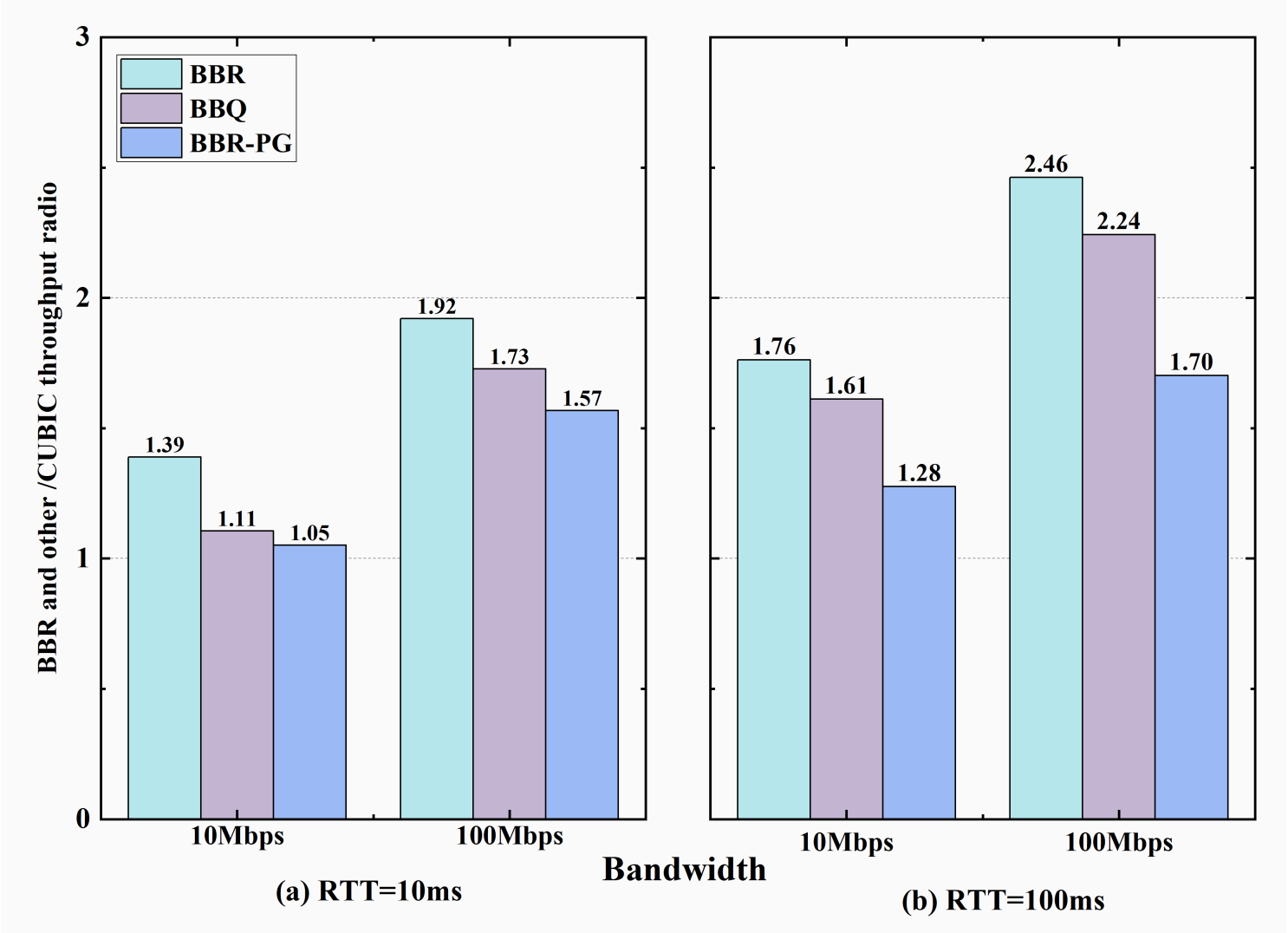


Figure 12

Throughput occupancy ratio of three algorithms when coexisting with CUBIC

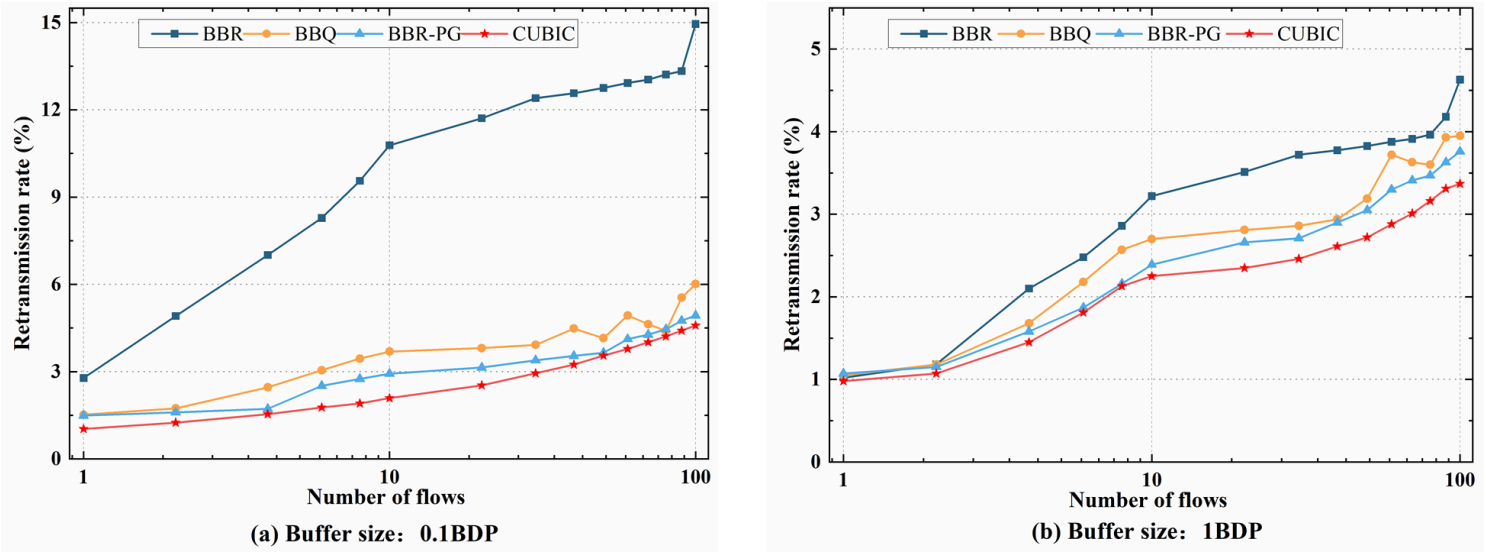


Figure 13

Retransmission rates of different numbers of flows in NS3

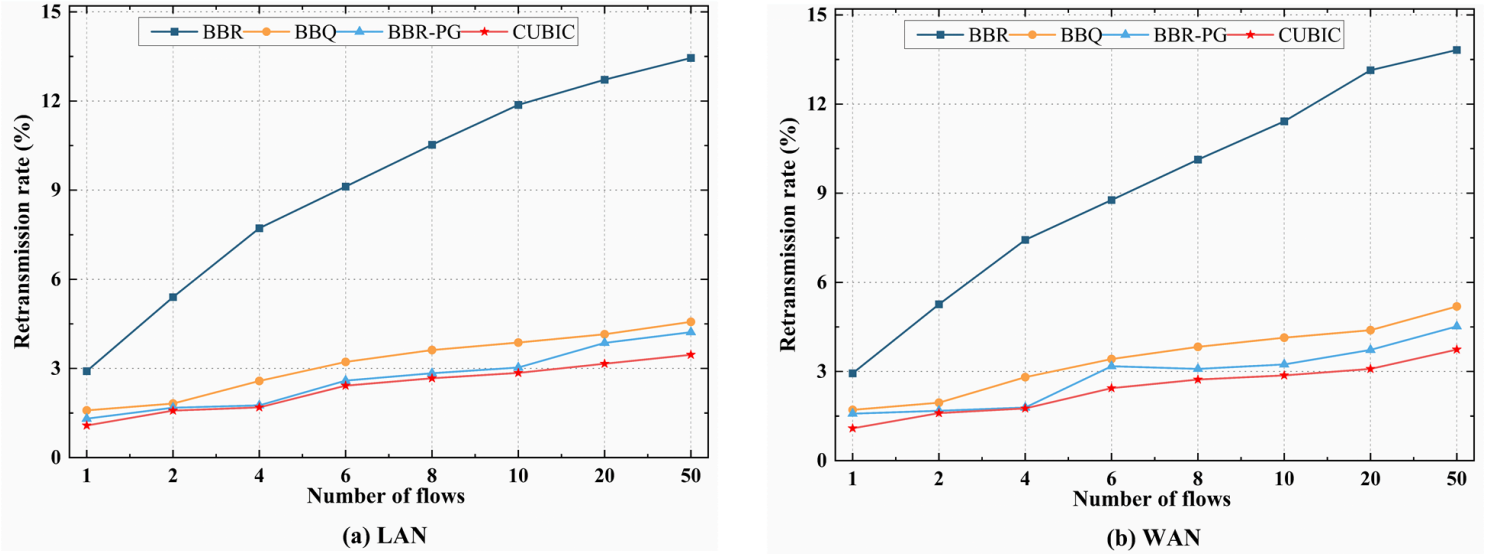


Figure 14

Retransmission rates of different numbers of flows on LAN and WAN networks